

REMONTIO

Dokumentacja techniczna

Spis treści

Informacje ogólne	4
Zakres Funkcjonalny Systemu	4
Harmonogramowanie Prac	4
Zarządzanie Finansami (Kosztorysy).....	4
Listy Zakupowe.....	4
Kalkulatory Budowlane.....	4
Dokumentacja Postępów (Foto)	5
Powiadomienia i Alerty	5
Funkcje Społecznościowe	5
Stack Technologiczny (Architektura).....	6
Wymagania dla środowiska deweloperskiego (Dev)	6
Backend (ASP.NET Core).....	6
Frontend (Vue.js).....	6
Baza Danych	7
Wymagania Sprzętowe (Developer).....	7
API	8
Informacje Ogólne.....	8
Endpointy	9
Auth i User	9
Project.....	10
Room	11
Budget	12
Tasks.....	13
ShoppingLists.....	14
Calculations	15
Contact i Alerts.....	15
Photos	16
DTO.....	17
User	17
Project.....	17
Room	18
Budget	19

Tasks	20
ShoppingLists	21
Calculations	21
Contacts	22
Alerts	22
Photos	23
Enum	23
StatusEnum.....	23
PriorityEnum	23
BudgetItemCategory	24
Baza danych	25
Opis tabel.....	25
Schemat bazy.....	26
Relacje i Integralność Danych.....	27
Procedury uruchomieniowe.....	28
Wymagania wstępne.....	28
Uruchomienie środowiska deweloperskiego.....	28
Ogólne	28
Backend	29
Frontend	29
Procedura publikacji.....	29
Backend.....	29
Frontend	30

Informacje ogólne

Typ aplikacji: Profesjonalna aplikacja webowa

Główny cel biznesowy: System dedykowany do kompleksowego wspomagania procesu zarządzania remontem. Aplikacja rozwiązuje problemy związane z chaosem informacyjnym i finansowym podczas prac budowlanych.

Zakres Funkcjonalny Systemu

System realizuje procesy biznesowe w następujących obszarach.

Harmonogramowanie Prac

Moduł odpowiedzialny za organizację czasu i przebiegu remontu

Tworzenie harmonogramów: Użytkownik może tworzyć plan ręcznie lub skorzystać z predefiniowanych szablonów

Edycja: Pełna możliwość modyfikacji istniejących harmonogramów

Zarządzanie Finansami (Kosztorysy)

Narzędzia do kontroli budżetu i wydatków.

Budżetowanie: Tworzenie budżetów remontowych

Kategoryzacja wydatków: Śledzenie kosztów z podziałem na materiały, robociznę oraz narzędzia

Etykietowanie: Możliwość oznaczania kosztorysów etykietami branżowymi (np. "hydraulika")

Eksport i Import: Eksport danych do formatów PDF i CSV oraz import kosztorysów z plików Excel

Listy Zakupowe

Wsparcie logistyki materiałowej

Tworzenie list: Dodawanie przedmiotów ręcznie lub automatycznie na podstawie wyników z kalkulatorów materiałowych

Organizacja: Oznaczanie list dedykowanymi etykietami

Kalkulatory Budowlane

Narzędzia wspomagające obliczanie zapotrzebowania na materiały

Kalkulator Farb/ Ścian: Obliczanie ilości farby na podstawie powierzchni i liczby warstw, z opcją dodania wyniku do listy zakupowej

Kalkulator Podłóg: Szacowanie ilości paneli/płytek z uwzględnieniem zapasu na docinki i odpady

Dokumentacja Postępów (Foto)

Archiwizacja: Dodawanie zdjęć dokumentujących stan przed, w trakcie i po remoncie

Katalogowanie: Przypisywanie zdjęć do konkretnych pomieszczeń i etapów prac

Powiadomienia i Alerty

System wspierający terminowość i kontrolę kosztów

Przypomnienia: Alerty o zaplanowanych pracach

Funkcje Społecznościowe

Baza kontaktów: Zarządzanie wizytówkami do fachowców

Stack Technologiczny (Architektura)

Aplikacja została zaprojektowana w architekturze SPA (Single Page Application) komunikującej się z bezstanowym REST API.

Obszar	Technologia	Opis / Rola
Backend	ASP.NET Core Web API	Główny framework aplikacji serwerowej.
Frontend	Vue.js 3	Framework warstwy prezentacji.
Baza Danych	Microsoft SQL Server	Relacyjna baza danych.
ORM	Entity Framework Core	Komunikacja z bazą danych.
State Mgmt	Pinia	Zarządzanie stanem aplikacji frontendowej.
Styling	Bootstrap 5 + SCSS	Biblioteka komponentów i stylowanie.

Wymagania dla środowiska deweloperskiego (Dev)

Aby uruchomić projekt lokalnie, stacja robocza programisty musi spełniać poniższe wymagania w podziale na Backend i Frontend.

Backend (ASP.NET Core)

System operacyjny:

- Windows 10/11
- Linux
- macOS

Wymagane oprogramowanie:

- **.NET SDK:** Wersja kompatybilna z ASP.NET Core (zalecana najnowsza stabilna wersja .NET 8).
- **IDE:** Visual Studio 2022 (zalecane) lub Visual Studio Code / JetBrains Rider

Zależności projektu:

- Identity (Autoryzacja)
- AutoMapper (Mapowanie obiektów DTO)
- FluentValidation (Walidacja danych wejściowych)

Frontend (Vue.js)

Środowisko uruchomieniowe: Node.js (wersja LTS)

IDE: Visual Studio Code / JetBrains Rider

Menedżer pakietów: npm

Narzędzia:

- NSwag Studio – do generowania klienta API i obsługi żądań.
- **Przeglądarka:** Najnowsze wersje Chrome, Firefox, Edge lub Safari.

Baza Danych

Lokalna instancja **Microsoft SQL Server**

Wymagania Sprzętowe (Developer)

Ze względu na użycie środowiska .NET, zalecane są następujące parametry stacji roboczej:

- **Minimalne:**
 - CPU: 2 rdzenie
 - RAM: 8 GB
 - Dysk: HDD
- **Zalecane:**
 - CPU: 4 rdzenie (Intel i5/i7 lub odpowiednik AMD)
 - RAM: 16 GB lub więcej (Docker + Visual Studio zużywają znaczne zasoby pamięci)
 - Dysk: SSD NVMe

API

System korzysta z architektury REST API. Komunikacja odbywa się w formacie JSON. API zostało zdefiniowane w standardzie OpenAPI 3.0.1 (Swagger).

Informacje Ogólne

Base URL: /api

Format danych: application/json (dla przesyłania plików: multipart/form-data).

Autoryzacja: Token Bearer (JWT). Token należy przesyłać w nagłówku Authorization każdego żądania do chronionych zasobów.

Endpointy

Poniżej zamieszczono każdy endpoint używany w aplikacji

Auth i User

Metoda	Endpoint	Opis	Input (Wejście)	Output (Wyjście)
POST	/api/User/log-in	Logowanie	username, password	UserDataDTO
POST	/api/User/register	Rejestracja	CreateUserDTO	CreateUserDTO
GET	/api/User/get-user	Dane użytkownika	id (GUID)	UserDataDTO
GET	/api/User/get-user-list	Lista użytkowników	-	List<UserDataDTO>
PUT	/api/User/update-user	Edycja profilu	id, Body: UserDataDTO	Boolean
PUT	/api/User/change-user-password	Zmiana hasła	email, oldPassword, newPassword	Boolean
PUT	/api/User/change-user-role	Zmiana roli	id, newRole	Boolean
DELETE	/api/User/delete-user	Usunięcie konta	id	Boolean
GET	/api/Token/generate-token	Generowanie tokena	Body: UserDataDTO	Status 200 OK
GET	/api/Token/get-refresh-token	Odświeżenie sesji	id, refreshToken	String (Token)

Project

Metoda	Endpoint	Opis	Input (Wejście)	Output (Wyjście)
POST	/api/Project/create-project	Nowy projekt	Body: CreateProjectDTO	Boolean
GET	/api/Project/get-project-by-id	Pobranie projektu	projectId	ProjectDataDTO
GET	/api/Project/get-project-list	Wszystkie projekty	-	List<ProjectDataDTO>
GET	/api/Project/get-project-list-by-user-id	Projekty użytkownika	Query: userId	List<ProjectDataDTO>
PUT	/api/Project/edit-project	Edycja projektu	Body: ProjectDataDTO	Boolean
PUT	/api/Project/edit-project-status	Zmiana statusu	projectId, status	Boolean
DELETE	/api/Project/delete-project	Usunięcie projektu	projectId	Boolean

Room

Metoda	Endpoint	Opis	Input (Wejście)	Output (Wyjście)
POST	/api/Room/create-room	Nowy pokój	projectId, Body: CreateRoomDTO	Boolean
GET	/api/Room/get-room-by-id	Pobranie pokoju	roomId	RoomDataDTO
GET	/api/Room/get-room-list-by-project-id	Pokoje w projekcie	projectId	List<RoomDataDTO>
PUT	/api/Room/edit-room	Edycja pokoju	Body: RoomDataDTO	Boolean
PUT	/api/Room/edit-room-status	Zmiana statusu	roomId, status	Boolean
DELETE	/api/Room/delete-room	Usunięcie pokoju	roomId	Boolean
POST	/api/Room/add-wall	Dodanie ściany	roomId, wallName, Body: List<PointDTO>	Boolean
DELETE	/api/Room/remove-wall	Usunięcie ściany	roomId, wallId	Boolean
GET	/api/Room/get-walls-by-room-id	Pobranie ścian	roomId	List<WallDTO>
POST	/api/Room/add-floor	Dodanie podłogi	roomId, floorName, Body: List<PointDTO>	Boolean
GET	/api/Room/get-floors-by-room-id	Pobranie podłóg	roomId	List<FloorDTO>

Budget

Metoda	Endpoint	Opis	Input (Wejście)	Output (Wyjście)
POST	/api/Budget/create-budget	Nowy budżet	Body: CreateBudgetDTO	Boolean
GET	/api/Budget/get-budget-by-id	Dane budżetu	budgetId	BudgetDataDTO
GET	/api/Budget/get-budget-list-by-project-id	Budżety w projekcie	projectId	List<BudgetDataDTO>
PUT	/api/Budget/edit-budget	Edycja nagłówka	Body: BudgetDataDTO	Boolean
POST	/api/Budget/add-budget-item	Dodanie kosztu	budgetId, Body: CreateBudgetItemDTO	Boolean
DELETE	/api/Budget/remove-budget-item	Usunięcie kosztu	budgetId, itemId	Boolean
PUT	/api/Budget/update-budget-item	Edycja kosztu	budgetId, Body: BudgetItemDataDTO	Boolean
PUT	/api/Budget/mark-item-completed	Status opłacenia	budgetId, itemId, isCompleted	Boolean
PUT	/api/Budget/budget-recalculate	Przeliczenie sumy	budgetId	Boolean
POST	/api/Budget/add-shopping-list-as-item	Import z listy zakupów	budgetId, shoppingListId, snapshot	Boolean

Tasks

Metoda	Endpoint	Opis	Input (Wejście)	Output (Wyjście)
POST	/api/Task/create-task	Nowe zadanie	Body: CreateTaskDTO	Boolean
GET	/api/Task/get-task-by-id	Szczegóły zadania	taskId	TaskDataDTO
GET	/api/Task/get-task-list-by-project-id	Zadania w projekcie	projectId	List<TaskDataDTO>
PUT	/api/Task/edit-task	Edycja zadania	Body: TaskDataDTO	Boolean
PUT	/api/Task/edit-task-status	Zmiana statusu	taskId, status	Boolean
PUT	/api/Task/edit-task-priority	Zmiana priorytetu	taskId, priority	Boolean
DELETE	/api/Task/delete-task	Usunięcie zadania	taskId	Boolean

ShoppingLists

Metoda	Endpoint	Opis	Input (Wejście)	Output (Wyjście)
POST	/api/List/create-list	Nowa lista	Body: CreateListDTO	Boolean
GET	/api/List/get-list-by-id	Pobranie listy	listId	ListDataDTO
GET	/api/List/get-list-by-project-id	Listy w projekcie	projectId	List<ListDataDTO>
PUT	/api/List/edit-list	Edycja nagłówka	Body: ListDataDTO	Boolean
GET	/api/List/get-item-list-by-list-id	Elementy listy	listId	List<ListItemDataDTO>
POST	/api/List/add-list-item	Dodanie produktu	listId, name, quantity, price	Boolean
DELETE	/api/List/remove-list-item	Usunięcie produktu	listId, itemId	Boolean
PUT	/api/List/mark-list-item-bought	Oznaczenie zakupu	listId, itemId, isBought	Boolean

Calculations

Metoda	Endpoint	Opis	Input (Wejście)	Output (Wyjście)
POST	/api/Calculations/create-calculation	Zapis kalkulacji	roomId, Body: CreateCalculationDTO	Boolean
GET	/api/Calculations/get-calculation-by-id	Pobranie kalkulacji	calculationId	CalculationDataDTO
GET	/api/Calculations/get-calculation-list-by-project-id	Kalkulacje w projekcie	projectId	List<CalculationDataDTO>
PUT	/api/Calculations/edit-calculation	Edycja wartości	Body: CalculationDataDTO	Boolean
PUT	/api/Calculations/edit-calculation-type	Zmiana typu	calculationId, type	Boolean

Contact i Alerts

Metoda	Endpoint	Opis	Input (Wejście)	Output (Wyjście)
POST	/api/Contact/create-contact	Nowy kontakt	Body: CreateContactDTO	Boolean
GET	/api/Contact/get-contact-list-by-user-id	Kontakty użytkownika	userId	List<ContactDataDTO>
PUT	/api/Contact/change-privacy	Zmiana widoczności	contactId, isPrivate	Boolean
POST	/api/Alert/create-alert	Nowe powiadomienie	Body: CreateAlertDTO	Boolean
GET	/api/Alert/get-alert-list-by-user-id	Powiadomienia usera	userId	List<AlertDataDTO>
PUT	/api/Alert/edit-alert-status	Zmiana statusu	alertId, status	Boolean

Photos

Metoda	Endpoint	Opis	Input (Wejście)	Output (Wyjście)
POST	/api/Photos/upload-photo	Wgranie zdjęcia	Form-Data: File (Binary), RoomId, ProjectId, Description, UserId	PhotoDataDTO
GET	/api/Photos/file	Pobranie pliku	photoId	Binary File / 200 OK
GET	/api/Photos/list-by-project	Lista zdjęć	projectId	List<PhotoDataDTO>
DELETE	/api/Photos/delete-photo	Usunięcie zdjęcia	photoId	Boolean

DTO

Poniżej zamieszczono każde DTO używane w aplikacji

User

```
"AuthAndUsers": {
  "CreateUserDTO": {
    "email": "string",
    "password": "string",
    "userName": "string",
    "name": "string",
    "surname": "string",
    "role": "string",
    "createdAt": "date-time"
  },
  "UserDataDTO": {
    "id": "uuid",
    "userName": "string",
    "name": "string",
    "surname": "string",
    "email": "string",
    "role": "string",
    "createdAt": "date-time"
  }
}
```

Project

```
"Projects": {
  "CreateProjectDTO": {
    "name": "string",
    "description": "string",
    "createdAt": "date-time",
    "status": "integer (StatusEnum)",
    "userId": "uuid"
  },
  "ProjectDataDTO": {
    "id": "uuid",
    "name": "string",
    "description": "string",
    "createAt": "date-time",
    "closedAt": "date-time",
    "status": "integer (StatusEnum)",
    "user": "UserDataDTO"
  }
}
```

Room

```
"Rooms": {
  "CreateRoomDTO": {
    "name": "string",
    "description": "string",
    "createdAt": "date-time",
    "closedAt": "date-time",
    "status": "integer (StatusEnum)",
    "userId": "uuid"
  },
  "RoomDataDTO": {
    "id": "uuid",
    "name": "string",
    "description": "string",
    "createAt": "date-time",
    "closedAt": "date-time",
    "status": "integer (StatusEnum)",
    "projectId": "uuid",
    "userId": "uuid",
    "taskIds": ["uuid"],
    "calculationIds": ["uuid"],
    "shoppingListIds": ["uuid"],
    "budgetIds": ["uuid"],
    "photoIds": ["uuid"],
    "wallIds": ["uuid"]
  },
  "WallDTO": {
    "id": "uuid",
    "name": "string",
    "calculatedArea": "double",
    "roomId": "uuid"
  },
  "FloorDTO": {
    "id": "uuid",
    "name": "string",
    "calculatedArea": "double",
    "roomId": "uuid"
  },
  "PointDTO": {
    "x": "float",
    "y": "float"
  }
}
```

Budget

```
"Budget": {
  "CreateBudgetDTO": {
    "name": "string",
    "description": "string",
    "createAt": "date-time",
    "total": "float",
    "spent": "float",
    "estimatedPrice": "float",
    "roomId": "uuid",
    "projectId": "uuid",
    "userId": "string"
  },
  "BudgetDataDTO": {
    "id": "uuid",
    "name": "string",
    "description": "string",
    "createAt": "date-time",
    "total": "float",
    "spent": "float",
    "estimatedPrice": "float",
    "itemIds": ["uuid"],
    "roomId": "uuid",
    "projectId": "uuid",
    "userId": "uuid"
  },
  "CreateBudgetItemDTO": {
    "name": "string",
    "description": "string",
    "category": "integer (BudgetItemCategoryEnum)",
    "price": "float",
    "total": "float",
    "estimatedPrice": "float",
    "isCompleted": "boolean"
  },
  "BudgetItemDataDTO": {
    "id": "uuid",
    "name": "string",
    "description": "string",
    "category": "integer (BudgetItemCategoryEnum)",
    "price": "float",
    "total": "float",
    "estimatedPrice": "float",
    "isCompleted": "boolean"
  }
}
```

Tasks

```
"Tasks": {
  "CreateTaskDTO": {
    "name": "string",
    "description": "string",
    "status": "integer (StatusEnum)",
    "priority": "integer (PriorityEnum)",
    "createAt": "date-time",
    "startAt": "date-time",
    "closedAt": "date-time",
    "estimatedTime": "date-time",
    "roomId": "uuid",
    "projectId": "uuid",
    "userId": "string"
  },
  "TaskDataDTO": {
    "id": "uuid",
    "name": "string",
    "description": "string",
    "status": "integer (StatusEnum)",
    "priority": "integer (PriorityEnum)",
    "createAt": "date-time",
    "startAt": "date-time",
    "closedAt": "date-time",
    "estimatedTime": "date-time",
    "roomId": "uuid",
    "projectId": "uuid",
    "userId": "uuid"
  }
}
```

ShoppingLists

```
"Lists": {
  "CreateListDTO": {
    "name": "string",
    "description": "string",
    "createAt": "date-time",
    "roomId": "uuid",
    "projectId": "uuid",
    "userId": "string"
  },
  "ListDataDTO": {
    "id": "uuid",
    "name": "string",
    "description": "string",
    "createAt": "date-time",
    "itemIds": ["uuid"],
    "roomId": "uuid",
    "projectId": "uuid",
    "userId": "uuid"
  },
  "ListItemDataDTO": {
    "id": "uuid",
    "name": "string",
    "quantity": "integer",
    "price": "float",
    "isBought": "boolean",
    "shoppingListId": "uuid"
  }
}
```

Calculations

```
"Calculations": {
  "CreateCalculationDTO": {
    "name": "string",
    "value": "float",
    "type": "integer (CalculationsTypeEnum)"
  },
  "CalculationDataDTO": {
    "id": "uuid",
    "name": "string",
    "value": "float",
    "type": "integer (CalculationsTypeEnum)",
    "roomId": "uuid",
    "projectId": "uuid",
    "userId": "uuid"
  }
}
```

Contacts

```
"Contacts": {
  "CreateContactDTO": {
    "name": "string",
    "description": "string",
    "contactDetails": "string",
    "createdDate": "date-time",
    "isPrivate": "boolean",
    "spec": "integer (SpecEnum)",
    "userId": "string"
  },
  "ContactDataDTO": {
    "id": "uuid",
    "name": "string",
    "description": "string",
    "contactDetails": "string",
    "createdDate": "date-time",
    "isPrivate": "boolean",
    "spec": "integer (SpecEnum)",
    "userId": "uuid"
  }
}
```

Alerts

```
"Alerts": {
  "CreateAlertDTO": {
    "message": "string",
    "startDate": "date-time",
    "endDate": "date-time",
    "userId": "uuid",
    "status": "string",
    "priority": "string"
  },
  "AlertDataDTO": {
    "id": "uuid",
    "message": "string",
    "createdAt": "date-time",
    "startDate": "date-time",
    "endDate": "date-time",
    "status": "string",
    "priority": "string",
    "userId": "uuid"
  }
}
```

Photos

```
"Photos": {
  "PhotoDataDTO": {
    "id": "uuid",
    "fileName": "string",
    "contentType": "string",
    "size": "integer (int64)",
    "url": "string",
    "storageProvider": "string",
    "createdAt": "date-time",
    "projectId": "uuid",
    "roomId": "uuid",
    "userId": "uuid"
  }
}
```

Enum

StatusEnum

```
"StatusEnum": {
  "values": {
    "0": "Planowany",
    "1": "W toku",
    "2": "Zakończony",
    "3": "Anulowany"
  }
}
```

PriorityEnum

```
"PriorityEnum": {
  "values": {
    "0": "Niski",
    "1": "Normalny",
    "2": "Wysoki",
    "3": "Krytyczny"
  }
}
```

BudgetItemCategory

```
"BudgetItemCategory": {
  "values": {
    "0": "Materiały",
    "1": "Robocizna",
    "2": "Narzędzia",
    "3": "Transport",
    "4": "Projekt i Dokumentacja",
    "5": "Wyposażenie",
    "6": "Media i Przyłącza",
    "7": "Formalności",
    "8": "Inne"
  }
}
```

CalculationsTypeEnum

```
"CalculationsTypeEnum": {
  "values": {
    "0": "Farba",
    "1": "Podłogi",
    "2": "Płytki",
    "3": "Tapety",
    "4": "Gładzie i Tynki"
  }
}
```

SpecEnum

```
"SpecEnum": {
  "values": {
    "0": "Ogólnobudowlana",
    "1": "Hydraulik",
    "2": "Elektryk",
    "3": "Malarz",
    "4": "Stolarz",
    "5": "Płytkarz",
    "6": "Murarz/Tynkarz",
    "7": "Architekt/Projektant"
  }
}
```


Baza danych

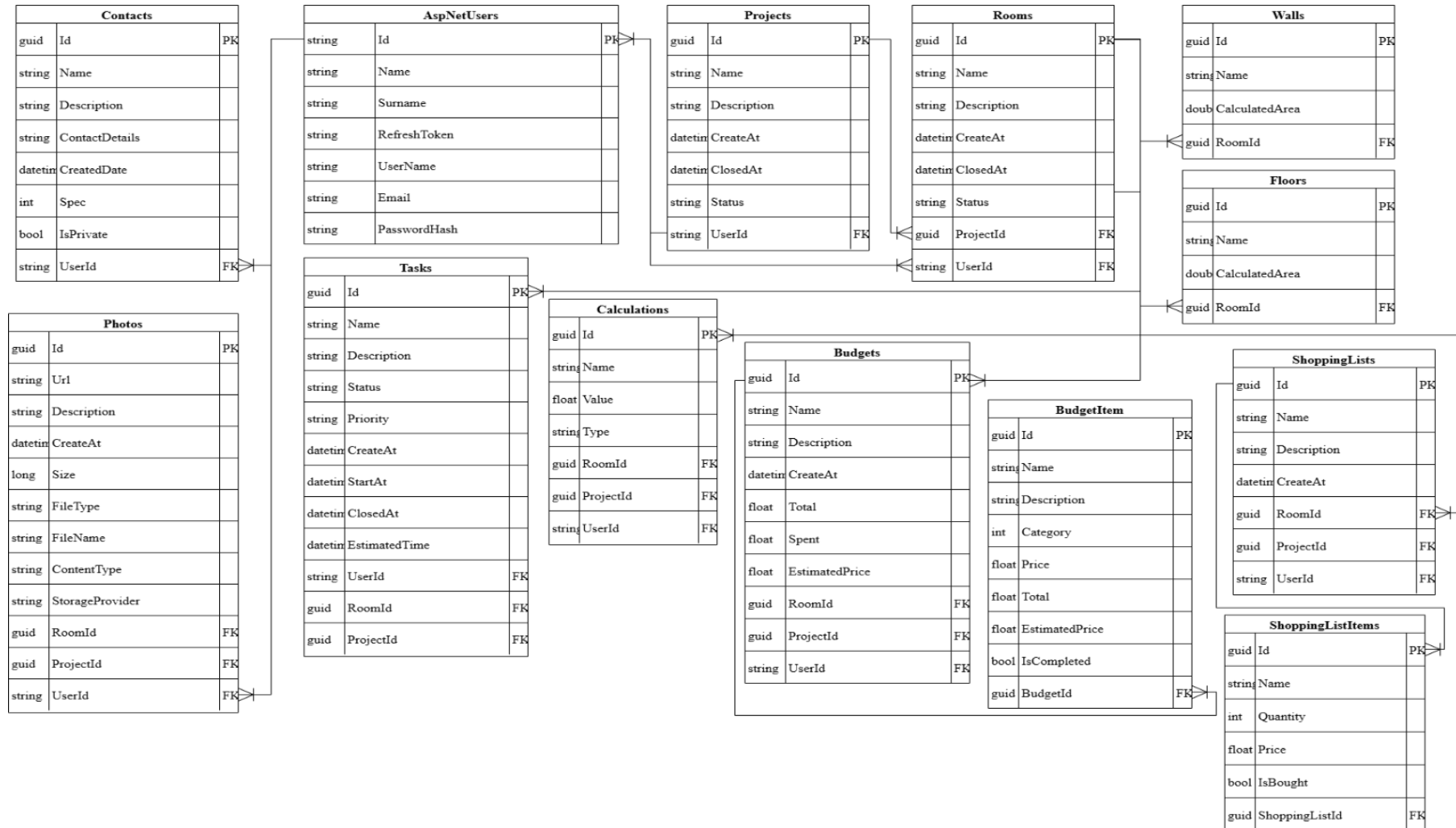
System **Remontio** korzysta z relacyjnej bazy danych **Microsoft SQL Server**. Struktura danych jest zarządzana poprzez **Entity Framework Core** w podejściu *Code-First*.

Opis tabel

LP	Nazwa tabeli	Opis i przechowywane dane	Przechowywane dane
1	AspNetUsers	Tabela pakietu Identity	.Przechowuje dane na temat użytkownika oraz tokeny bezpieczeństwa
2	Project	Główna tabela łącząca dane o remoncie.	Przechowuje nazwę projektu, opis, daty rozpoczęcia i zakończenia oraz status realizacji
3	Rooms	Definiuje pomieszczenia wchodzące w skład projektu. Jest punktem odniesienia dla ścian oraz podłóg	Posiada nazwę, opis, daty rozpoczęcia i zakończenia, priorytet wykonania oraz status realizacji
4	Walls / Floors	Tabele przechowujące wyliczoną powierzchnię elementu	Przechowuje nazwę i wyliczoną powierzchnię
5	Calculations	Magazynuje wyniki obliczeń materiałowych powiązanych z projektem i pokojem	Nazwa, wartość, typ
6	Budgets	Określa finanse projektu	Zawiera nazwę, opis oraz sumaryczne informacje o kosztach - koszt całkowity, wydatki
7	BudgetItem	Szczegółowe pozycje budżetu	Znajduje się w niej nazwa, opis, kategoria, cena szacowana oraz cena całkowita
8	ShoppingLists	Tabela nagłówkowa agregująca produkty do zakupienia w ramach budżetu	Nazwa, opis
9	ShoppingListItem	Konkretne produkty do zakupienia	Nazwa, opis, cena, ilość i informację czy już kupione
10	Photos	Tabela zawierająca informacje o zdjęciach	Przechowuje URL, opis, typ danych, nazwa pliku
11	Contacts	Baza kontaktów do osób wprowadzonych przez użytkownika	Posiada nazwę, opis oraz dane kontaktowe

Schemat bazy

Poniższy schemat przedstawia relacje pomiędzy encjami w systemie. Centralnym punktem jest tabela Projects oraz Rooms, do których podpięte są pozostałe moduły funkcjonalne.



Relacje i Integralność Danych

- **Relacja One-to-Many (Jeden do Wiele):**
 - User -> Projects (Użytkownik ma wiele projektów).
 - Project -> Rooms (Projekt ma wiele pokoi).
 - Room -> Tasks, Budgets, Photos (Pokój agreguje zadania, koszty i zdjęcia).
 - Budget -> BudgetItems (Budżet składa się z wielu pozycji).
- **Klucze Obce (Foreign Keys):** Wszystkie tabele podrzędne posiadają klucz UserId, co ułatwia szybkie filtrowanie danych per użytkownik bez konieczności wykonywania wielu złączeń (JOIN) przez tabelę Projects.
- **Typy Danych:**
 - Kwoty pieniężne (Price, Total) przechowywane są jako float (w środowisku produkcyjnym zalecana migracja na decimal).
 - Identyfikatory to GUID (Global Unique Identifier), co ułatwia skalowanie i unikalność danych przy ewentualnej migracji danych.

Procedury uruchomieniowe

Niniejszy rozdział opisuje proces konfiguracji środowiska lokalnego (Development), uruchamiania testów

Wymagania wstępne

Przed przystąpieniem do pracy upewnij się, że na stacji roboczej zainstalowane są:

- **Git** (system kontroli wersji)
- **Visual Studio 2022** lub **VS Code** (zalecane środowiska IDE).
- **.NET SDK 8.0+** (zgodnie z wersją projektu backendowego)
- **Node.js** (wersja LTS) + **npm** (dla Frontendu)
- **Microsoft SQL Server** (wersja Express lub Developer) LUB **LocalDB** (często instalowany z Visual Studio).

Uruchomienie środowiska deweloperskiego

Proces uruchamiania został podzielony na Backend i Frontend.

Ogólne

Krok 1: Pobranie repozytorium

```
git clone <adres-twojego-repozytorium>
cd remontio-project
```

Krok 2: Skonfiguruj bazę danych

Connection String: Otwórz plik appsettings.json (lub appsettings.Development.json) w projekcie Backendowym. Upewnij się, że sekcja ConnectionStrings wskazuje na Twój lokalny serwer.

Przykład dla LocalDB:

```
"ConnectionStrings": { "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=RemontioDb;Trusted_Connection=True;MultipleActiveResultSets=true" }
```

Przykład dla SQL Server Express:

```
"ConnectionStrings": {
  "DefaultConnection": "Server=.\SQLEXPRESS;Database=RemontioDb;Trusted_Connection=True;TrustServerCertificate=True;"
}
```

Krok 3: Aplikacja migracji

Projekt wykorzystuje podejście Code-First. Aby utworzyć bazę danych i tabele, wykonaj w terminalu (w katalogu projektu API):

```
dotnet tool install --global dotnet-ef # Tylko jeśli nie zainstalowano wcześniej  
dotnet restore  
dotnet ef database update
```

Backend

Krok 1: Przejdź w terminalu do katalogu głównego API (tam gdzie plik .csproj lub .sln).

Krok 2: Uruchom aplikację:

```
dotnet run
```

Krok 3: API powinno wystartować pod adresem <https://localhost:5001> (lub 7XXX - sprawdź w konsoli). Sprawdź dostępność dokumentacji pod adresem /swagger.

Frontend

Krok 1: Przejdź w terminalu do katalogu z kodem frontendowym.

Krok 2: Zainstaluj brakujące biblioteki (tylko przy pierwszym uruchomieniu):

```
npm install
```

Krok 3: Sprawdź plik .env (lub .env.development). Upewnij się, że zmienna wskazująca na API (np. VITE_API_URL) ma ten sam port, na którym uruchomił się backend.

```
VITE_API_URL=https://localhost:5001/api
```

Krok 4: Uruchom serwer deweloperski:

```
npm run dev
```

Krok 5: Aplikacja będzie dostępna w przeglądarce (zazwyczaj <http://localhost:5173>).

Procedura publikacji

Wdrożenie na środowisko produkcyjne odbywa się poprzez publikację kodu (Code Deploy).

Backend

Aby wygenerować pliki gotowe do wysłania na serwer:

```
dotnet publish -c Release -o ./publish
```

Wygenerowaną zawartość folderu ./publish należy wgrać na serwer.

Frontend

Aby zbudować zoptymalizowaną wersję produkcyjną aplikacji Vue:

```
npm run build
```

Wynikiem będzie folder dist, który należy serwować jako pliki statyczne.